

HuggingFace를 활용한
인공지능 웹 애플리케이션 개발

Chapter 7

멀티모달 프로젝트 따라하기

CLIP · LLaVA · MiniGPT-4 · FastAPI · 멀티모달 챗봇 · 이미지 검색

2025년판 개정증보

목 차

7장 서문	3
7.1 텍스트+이미지 입력 모델 실습 (CLIP)	4
(1) CLIP의 개념 및 작동 원리	4
(2) openai/clip-vit-base-patch32 모델 실습 환경 구성	7
(3) 이미지 벡터 생성 및 텍스트 벡터 생성	9
(4) 코사인 유사도 계산 → 이미지 검색 구현	12
7.2 멀티모달 챗봇 만들기 (MiniGPT-4 & LLaVA)	16
(1) MiniGPT-4 / LLaVA 개요 및 모델 구조	16
(2) 비전+텍스트 통합 프롬프트 설계	19
(3) 경량화 모델 및 Hugging Face Spaces 활용	22
(4) FastAPI 챗봇 API: 이미지+질문 → 응답	24
7.3 FastAPI 기반 멀티모달 API 구성하기	27
(1) 이미지+텍스트를 함께 POST로 전송하는 방법	27
(2) UploadFile + Form 조합 처리	29
(3) FormData → 모델 입력 전처리 파이프라인	31
(4) 추론 결과 + 텍스트 응답 결합 구조	33
(5) Swagger UI에서 이미지 업로드+프롬프트 테스트	35
7.4 업로드 + 프롬프트 통합 처리	37
(1) 업로드 이미지 처리: 임시 저장 vs 메모리 처리	37
(2) 자연어 프롬프트 처리 전략	39
(3) 통합 처리 흐름: 업로드 → 전처리 → 추론 → 응답	41
(4) Swagger UI / Postman / Streamlit 테스트 방법	44
7장 마무리 및 연습 문제	47

7장 멀티모달 프로젝트 따라하기

이 장에서는 멀티모달(Multimodal) 인공지능 기술을 FastAPI 기반의 실제 웹 프로젝트로 구현합니다. 멀티모달 AI는 텍스트, 이미지, 음성, 비디오 등 두 가지 이상의 데이터 형태를 동시에 이해하고 처리하는 기술로, 현재 가장 빠르게 발전하고 있는 AI 분야입니다.

단일 모달리티(Single-Modality) 모델이 텍스트 또는 이미지 중 하나만 처리하는 데 반해, 멀티모달 모델은 "이 사진 속 인물이 무엇을 하고 있나요?", "이 차트가 의미하는 바를 설명해줘" 같은 질문을 자연스럽게 처리합니다. ChatGPT Vision, Google Gemini, Anthropic Claude의 비전 기능이 모두 이 멀티모달 기술의 산물입니다.

7.1절에서는 CLIP 모델로 텍스트-이미지 공동 임베딩 공간을 이해하고 이미지 검색을 구현합니다. 7.2절에서는 MiniGPT-4와 LLaVA를 활용한 멀티모달 챗봇을 FastAPI로 구축합니다. 7.3절에서는 이미지와 텍스트를 함께 처리하는 REST API의 완전한 구조를 설계하고, 7.4절에서는 파일 업로드부터 자연어 응답까지의 통합 처리 흐름을 마스터합니다.

이 장에서 배울 내용

- CLIP으로 이미지-텍스트 유사도 계산 및 시맨틱 이미지 검색 구현
- LLaVA / MiniGPT-4로 "이미지를 보고 답변하는" 멀티모달 챗봇 구축
- FastAPI에서 이미지(UploadFile)와 텍스트(Form)를 함께 처리하는 멀티모달 API 설계
- 메모리 처리 vs 임시 저장: 파일 업로드 전략 비교 및 실습
- Swagger UI, Postman, Streamlit으로 멀티모달 API 테스트하는 방법

실습 환경 요구 사항

- Python 3.9 이상

- `pip install transformers torch torchvision Pillow fastapi uvicorn`
- `pip install python-multipart open_clip_torch accelerate`
- GPU (VRAM 8GB 이상 권장) - LLaVA 실행 시 필요
- Hugging Face 계정 (일부 모델 다운로드 시 필요)
- Google Colab Pro 또는 로컬 GPU 환경 권장

7.1 텍스트+이미지 입력 모델 실습 (CLIP)

(1) CLIP(OpenAI): 이미지와 텍스트를 같은 벡터 공간에 매핑

CLIP(Contrastive Language- Image Pre- Training)은 2021년 OpenAI가 발표한 혁신적인 멀티모달 모델입니다. CLIP은 이미지 인코더(Image Encoder)와 텍스트 인코더(Text Encoder)를 동시에 학습하여, 이미지와 텍스트를 동일한 고차원 벡터 공간(shared embedding space)에 투영합니다. 이를 통해 이미지와 텍스트 간의 의미적 유사성을 직접 비교할 수 있습니다.

핵심 아이디어는 "의미적으로 일치하는 이미지-텍스트 쌍은 벡터 공간에서 가깝게, 관계없는 쌍은 멀리 위치하도록" 대조 학습(Contrastive Learning)을 수행하는 것입니다. 4억 개의 (이미지, 텍스트) 쌍 데이터로 학습되었으며, 이미지 분류, 이미지 검색, 시각적 질의응답 등 다양한 다운스트림 태스크에 활용됩니다.

구성 요소	구조	입력	출력	역할
Image Encoder	ViT (Vision Transformer) 또는 ResNet	PIL Image (224× 224)	512/768차원 벡터	이미지를 의미 벡터로 변환
Text Encoder	GPT-2 유사 Transformer	텍스트 문자열 (max 77 tokens)	512/768차원 벡터	텍스트를 의미 벡터로 변환
Projection Layer	선형 변환 레이어	각 인코더 출력	공통 임베딩 공간 벡터	두 벡터를 같은 공간으로 정렬

CLIP의 학습 방식인 대조 학습을 좀 더 자세히 살펴봅니다. 배치(batch) 내 N개의 이미지-텍스트 쌍이 있을 때, 대각선에 위치한 N개의 일치 쌍의 코사인 유사도는 최대화하고, 나머지 $N^2 - N$ 개의 불일치 쌍의 유사도는 최소화하도록 학습됩니다.

▶ CLIP 대조 학습 (Contrastive Learning) 개념

학습 데이터 예시 (3쌍):

이미지 1: 고양이 사진 텍스트 1: "a photo of a cat"
 이미지 2: 강아지 사진 텍스트 2: "a cute dog playing"
 이미지 3: 자동차 사진 텍스트 3: "a red sports car"

학습 목표 (유사도 행렬):

 텍스트1 텍스트2 텍스트3
 이미지1 → [높음] [낮음] [낮음] ← 대각선 최대화
 이미지2 → [낮음] [높음] [낮음]
 이미지3 → [낮음] [낮음] [높음]

이렇게 학습하면 CLIP은 의미가 일치하는 이미지-텍스트 쌍을
 벡터 공간에서 가까운 위치에 배치하는 능력을 획득합니다.

CLIP이 실용적으로 강력한 이유 중 하나는 Zero-shot 전이 학습(Zero-shot Transfer) 능력입니다.
 별도의 파인튜닝 없이도 텍스트 레이블만 작성하면 새로운 분류 태스크를 즉시 수행할 수 있습니다.

CLIP 활용 태스크	방식	실용 예시
이미지 분류	이미지 벡터 vs. "a photo of [class]" 텍스트 벡터 비교	1000개 클래스 분류, Zero-shot ImageNet
시맨틱 이미지 검색	텍스트 쿼리 벡터로 이미지 DB를 검색	"아이와 함께 있는 개" 검색
이미지 캡셔닝 지원	여러 캡션 후보 중 이미지와 가장 유사한 것 선택	BLIP 등 캡셔닝 모델과 결합
이미지 생성 가이드	생성된 이미지가 프롬프트와 얼마나 일치하는지 점수화	Stable Diffusion의 CLIP Score 측정
멀티모달 검색엔진	텍스트와 이미지를 동시에 인덱싱 → 크로스모달 검색	사진 앱의 자연어 검색 기능

(2) openai/clip-vit-base-patch32 모델 실습 환경 구성

CLIP 모델은 Hugging Face Hub에서 transformers 라이브러리를 통해 간편하게 로드할 수 있습니다. 가장 널리 사용되는 clip-vit-base-patch32는 ViT-B/32 이미지 인코더와 GPT-2 스타일의 텍스트 인코더를 사용하며 512차원 임베딩 벡터를 생성합니다.

모델 ID	이미지 인코더	임베딩 차원	VRAM	특징
openai/clip-vit-base-patch32	ViT-B/32	512	~ 1.5GB	가장 빠름, 경량. 본 모델
openai/clip-vit-base-patch16	ViT-B/16	512	~ 1.5GB	patch32보다 세밀한 이미지 인식
openai/clip-vit-large-patch14	ViT-L/14	768	~ 3.5GB	고품질, Stable Diffusion에서 사용
laion/CLIP-ViT-H-14-laion2B-s32B-b79K	ViT-H/14	1024	~ 8GB	최고 성능, 대규모 데이터셋

```
# —— 환경 설치 ——
# pip install transformers torch torchvision Pillow open_clip_torch

# —— 라이브러리 및 모델 로드 ——
import torch
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import requests
from io import BytesIO

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"실행 장치: {device}")

model_id = "openai/clip-vit-base-patch32"

# CLIPProcessor: 이미지와 텍스트를 모델 입력 형식으로 변환
processor = CLIPProcessor.from_pretrained(model_id)

# CLIPModel: 이미지 인코더 + 텍스트 인코더 통합 모델
model = CLIPModel.from_pretrained(model_id).to(device)
```

```
model.eval()

print("CLIP 모델 로드 완료")
print(f"이미지 임베딩 차원: {model.config.projection_dim}")
# 출력: 이미지 임베딩 차원: 512

# —— 모델 파라미터 수 확인 ——
total_params = sum(p.numel() for p in model.parameters())
print(f"총 파라미터 수: {total_params:,} ({total_params/1e6:.1f}M)")
# 출력: 총 파라미터 수: 151,277,313 (151.3M)
```

[코드 7-1] CLIP 모델 로드 및 기본 정보 확인

💡 CLIPProcessor는 이미지 전처리(리사이즈, 정규화)와 텍스트 토크나이징을 모두 담당합니다. 이 이미지와 텍스트를 별도의 전처리 과정 없이 `processor(images=[img], text=["..."], return_tensors="pt")` 한 줄로 처리할 수 있습니다.

(3) 이미지 입력 → 벡터 생성 / 텍스트 입력 → 벡터 생성

CLIP의 이미지 인코더와 텍스트 인코더를 개별적으로 사용하여 각각의 임베딩 벡터를 생성하는 방법을 실습합니다. 이 임베딩 벡터는 코사인 유사도 계산, 검색 인덱스 구축, 분류 등 다양한 하위 태스크의 기반이 됩니다.

```
import torch
import torch.nn.functional as F
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import numpy as np

processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
model.eval()

#
# 이미지 임베딩 생성
#
def encode_image(image_input, normalize: bool = True) -> torch.Tensor:
    """
    PIL Image 또는 파일 경로로부터 CLIP 이미지 임베딩 벡터를 생성합니다.
    Args:
        image_input: PIL.Image 객체 또는 이미지 파일 경로 문자열
        normalize: True이면 L2 정규화 적용 (코사인 유사도 계산 시 내적으로 대체 가능)
    Returns:
        shape [1, 512] 의 임베딩 텐서
    """
    if isinstance(image_input, str):
        image_input = Image.open(image_input).convert("RGB")

    # CLIPProcessor로 이미지 전처리
    # - 224×224 리사이즈
    # - RGB 정규화 (mean=[0.48145, 0.4578, 0.40821], std=[...])
    inputs = processor(images=image_input, return_tensors="pt")
```

```

with torch.no_grad():
    # get_image_features(): 이미지 인코더 출력 → Projection Layer 통과
    image_features = model.get_image_features(**inputs)

    if normalize:
        image_features = F.normalize(image_features, p=2, dim=-1)

    return image_features

#
# 텍스트 임베딩 생성
#
def encode_text(texts, normalize: bool = True) -> torch.Tensor:
    """
    텍스트 문자열 또는 리스트로부터 CLIP 텍스트 임베딩 벡터를 생성합니다.
    Args:
        texts: 문자열 또는 문자열 리스트
        normalize: True이면 L2 정규화 적용
    Returns:
        shape [N, 512] 의 임베딩 텐서 (N = 텍스트 수)
    """
    if isinstance(texts, str):
        texts = [texts]

    # CLIPProcessor로 텍스트 토크나이징
    # max_length=77, padding=True, truncation=True 자동 적용
    inputs = processor(text=texts, return_tensors="pt", padding=True, truncation=True)

    with torch.no_grad():
        # get_text_features(): 텍스트 인코더 출력 → Projection Layer 통과
        text_features = model.get_text_features(**inputs)

        if normalize:
            text_features = F.normalize(text_features, p=2, dim=-1)

    return text_features

```

[코드 7-2] CLIP 이미지 및 텍스트 임베딩 생성 함수

```
# —— 테스트 실행

from PIL import Image
import numpy as np

# 테스트 이미지 생성 (실제 사용 시 실제 이미지 경로 사용)
# Image.open("your_image.jpg") 형태로 사용
test_image = Image.new("RGB", (224, 224), color=(100, 150, 200)) # 임시 파란색 이미지

# 이미지 임베딩 생성
img_embedding = encode_image(test_image)
print(f"이미지 임베딩 shape: {img_embedding.shape}")
print(f"이미지 임베딩 크기(L2 norm): {img_embedding.norm(dim=-1).item():.4f}")
# 출력:
# 이미지 임베딩 shape: torch.Size([1, 512])
# 이미지 임베딩 크기(L2 norm): 1.0000 (정규화 후)

# 텍스트 임베딩 생성
texts = [
    "a photo of a cat",
    "a blue sky with clouds",
    "a person riding a bicycle",
    "a bowl of ramen noodles",
]

text_embeddings = encode_text(texts)
print(f"텍스트 임베딩 shape: {text_embeddings.shape}")
# 출력: 텍스트 임베딩 shape: torch.Size([4, 512])

# 첫 번째 텍스트 임베딩 상위 5차원 값 확인
print("첫 텍스트 임베딩 (앞 5차원):", text_embeddings[0, :5].tolist())
# 출력: [0.0234, -0.0187, 0.0312, -0.0089, 0.0156] (예시)
```

[코드 7-3] 임베딩 생성 테스트 및 shape 확인

(4) 벡터 간 코사인 유사도 계산 → 가장 잘 맞는 이미지 찾기

L2 정규화된 벡터 간의 코사인 유사도는 단순한 내적(dot product) 연산으로 계산됩니다. 코사인 유사도는 -1에서 1 사이의 값으로, 1에 가까울수록 의미적으로 유사하다는 것을 의미합니다.

```
import torch
import torch.nn.functional as F
from PIL import Image
import os

#
# 코사인 유사도 계산 함수
#
def cosine_similarity(vec_a: torch.Tensor, vec_b: torch.Tensor) -> torch.Tensor:
    """L2 정규화된 벡터 간 코사인 유사도 = 내적 (dot product)"""
    # vec_a: [1, 512], vec_b: [N, 512]
    # 결과: [1, N] - 각 vec_b에 대한 유사도 점수
    return torch.mm(vec_a, vec_b.T)

#
# 이미지 검색 시스템 구현
#
class CLIPImageSearcher:
    """
    CLIP 기반 시맨틱 이미지 검색 클래스.
    이미지 컬렉션을 인덱싱하고 텍스트 쿼리로 검색합니다.
    """
    def __init__(self):
        self.image_embeddings = None # shape: [N, 512]
        self.image_paths = [] # 이미지 경로 리스트
        self.image_metadata = [] # 추가 메타데이터 (선택)

    def index_images(self, image_paths: list, batch_size: int = 16):
        """이미지 목록을 배치 단위로 임베딩하여 인덱스 구축"""
        all_embeddings = []
        valid_paths = []
```

```

for i in range(0, len(image_paths), batch_size):
    batch_paths = image_paths[i:i + batch_size]
    batch_images = []

    for path in batch_paths:
        try:
            img = Image.open(path).convert("RGB")
            batch_images.append(img)
            valid_paths.append(path)
        except Exception as e:
            print(f"이미지 로드 실패: {path} - {e}")
            continue

    if not batch_images:
        continue

    # 배치 처리로 효율적 임베딩
    inputs = processor(images=batch_images, return_tensors="pt", padding=True)
    with torch.no_grad():
        embeddings = model.get_image_features(**inputs)
        embeddings = F.normalize(embeddings, p=2, dim=-1)

    all_embeddings.append(embeddings.cpu())
    print(f"인덱싱 진행: {min(i+batch_size, len(image_paths))}/{len(image_paths)}")

    if all_embeddings:
        self.image_embeddings = torch.cat(all_embeddings, dim=0)
        self.image_paths = valid_paths
        print(f"인덱스 구축 완료: {len(self.image_paths)}장 이미지")
    else:
        raise ValueError("유효한 이미지가 없습니다.")

def search(self, query_text: str, top_k: int = 5) -> list:
    """텍스트 쿼리로 가장 유사한 이미지를 검색"""
    if self.image_embeddings is None:
        raise RuntimeError("먼저 index_images()를 실행하세요.")

```

```

# 텍스트 쿼리 임베딩
text_embedding = encode_text(query_text) # [1, 512]

# 코사인 유사도 계산 [1, N]
similarities = cosine_similarity(text_embedding.cpu(), self.image_embeddings)
similarities = similarities.squeeze(0) # [N]

# 상위 top_k 결과 추출
top_scores, top_indices = similarities.topk(min(top_k, len(self.image_paths)))

results = []
for score, idx in zip(top_scores.tolist(), top_indices.tolist()):
    results.append({
        "rank": len(results) + 1,
        "image_path": self.image_paths[idx],
        "similarity_score": round(score, 4),
        "similarity_pct": f"{score * 100:.1f}%"
    })

return results

def search_by_image(self, query_image, top_k: int = 5) -> list:
    """이미지로 유사한 이미지를 검색 (이미지-to-이미지 검색)"""
    img_embedding = encode_image(query_image) # [1, 512]
    similarities = cosine_similarity(img_embedding.cpu(), self.image_embeddings)
    similarities = similarities.squeeze(0)
    top_scores, top_indices = similarities.topk(min(top_k + 1, len(self.image_paths)))

    results = []
    for score, idx in zip(top_scores.tolist(), top_indices.tolist()):
        results.append({
            "rank": len(results) + 1,
            "image_path": self.image_paths[idx],
            "similarity_score": round(score, 4),
        })

    return results[1:] # 자기 자신 제외

```

[코드 7-4] CLIP 기반 시맨틱 이미지 검색 클래스 전체 구현

```

# —— 검색 시스템 사용 예시 ——
import glob

# 이미지 인덱스 구축
searcher = CLIPImageSearcher()

# 현재 디렉토리의 모든 JPG/PNG 파일 인덱싱
image_files = glob.glob("./images/**/*.*jpg", recursive=True)
image_files += glob.glob("./images/**/*.*png", recursive=True)

searcher.index_images(image_files, batch_size=32)

# 텍스트 쿼리로 검색
query = "a dog playing in the park"
results = searcher.search(query, top_k=5)

print(f"쿼리: {query}에 대한 검색 결과 TOP 5")
print("-" * 55)
for r in results:
    print(f" {r['rank']}위 {r['similarity_pct']:6s} {r['image_path']}")

# —— CLIP으로 Zero-shot 이미지 분류 ——
def zero_shot_classify(image, candidate_labels: list) -> dict:
    """레이블 목록을 기반으로 Zero-shot 이미지 분류 수행"""
    img_emb = encode_image(image) # [1, 512]
    txt_emb = encode_text(candidate_labels) # [N, 512]

    # 유사도 계산 후 softmax로 확률 변환
    logit_scale = model.logit_scale.exp() # CLIP 내부 스케일 팩터
    logits = logit_scale * cosine_similarity(img_emb, txt_emb) # [1, N]
    probs = logits.softmax(dim=-1).squeeze(0)

    return {label: round(prob.item(), 4)
            for label, prob in zip(candidate_labels, probs)}

# 테스트

```

```
test_image = Image.open("./test_dog.jpg")
labels = ["a cat", "a dog", "a bird", "a fish", "a car"]
results = zero_shot_classify(test_image, labels)

print("Zero-shot 분류 결과:")
for label, prob in sorted(results.items(), key=lambda x: -x[1]):
    bar = " " * int(prob * 30)
    print(f" {label:15s} {bar:30s} {prob:.2%}")
```

[코드 7-5] 이미지 검색 실행 예시 및 Zero-shot 분류

7.2 멀티모달 챗봇 만들기 (MiniGPT-4 & LLaVA)

(1) MiniGPT-4 / LLaVA 개요 및 모델 구조

MiniGPT-4와 LLaVA는 비전 인코더(Vision Encoder)를 대형 언어 모델(LLM)에 연결하는 멀티모달 대화 모델입니다. 이미지를 입력받아 자연어로 상세히 설명하거나, 이미지에 대한 질문에 답하거나, 이미지 기반 추론을 수행할 수 있습니다. 이는 GPT-4 Vision, Google Gemini Vision과 유사한 오픈소스 구현입니다.

모델	비전 인코더	언어 모델	연결 방식	
LLaVA-1.5	CLIP ViT-L/14	Vicuna-7B/13B	MLP Projection	가장 널리 사용
LLaVA-1.6 (Next)	CLIP ViT-L/14 (고해상도)	Mistral-7B/Qwen	MLP (고해상도 지원)	고해상도 이미지
MiniGPT-4	EVA-ViT-G	Vicuna-13B	Q-Former (BLIP-2)	최초의 오픈소스 모델
InternVL	InternViT-6B	InternLM-2	동적 고해상도 타일링	고해상도, 다
Qwen-VL	ViT-bigG	Qwen-7B	Position-aware VQA	알리바바 개발
moondream2	SigLIP	Phi-1.5 (1.8B)	MLP	초경량(~2GB)

LLaVA-1.5의 전체 아키텍처 흐름을 살펴봅니다. 핵심은 비전 인코더의 출력(이미지 패치 임베딩)을 LLM이 이해할 수 있는 텍스트 토큰과 동일한 공간으로 투영하는 MLP Projection입니다.

🔗 LLaVA-1.5 처리 흐름 (이미지+텍스트 → 응답 생성)

① 이미지 입력: 사용자 이미지 (임의 크기) → 336×336 리사이즈

② 비전 인코딩: CLIP ViT-L/14 → 256개 이미지 패치 임베딩 생성

각 패치: 1024차원 벡터

③ 투영 변환: MLP Projection (2-layer) → 256개 패치 × 4096차원

(LLM의 토큰 임베딩 차원과 동일하게 맞춤)

④ 토큰 시퀀스 구성: [시스템 프롬프트 토큰] + [이미지 토큰 256개] + [질문 토큰]

⑤ LLM 추론: Vicuna-7B가 이 전체 시퀀스를 처리하여 응답 텍스트 자동 생성

⑥ 출력: 자연어 응답 텍스트 (스트리밍 또는 완성 후 반환)

(2) "이미지를 보고 설명해줘" → 비전+텍스트 통합 프롬프트 설계

LLaVA 계열 모델은 특정 프롬프트 형식(Prompt Template)을 따릅니다. 모델마다 최적 프롬프트 형식이 다르며, 이를 올바르게 사용하지 않으면 응답 품질이 크게 저하됩니다. Hugging Face transformers의 `apply_chat_template()`을 사용하면 모델별 최적 형식이 자동 적용됩니다.

```
from transformers import LlavaNextProcessor, LlavaNextForConditionalGeneration
import torch
from PIL import Image

# —— LLaVA-1.6 (LLaVA-Next) 로드 ——
# 참고: 첫 다운로드 시 ~16GB 소요 (7B 모델)
model_id = "llava-hf/llava-v1.6-mistral-7b-hf"

processor = LlavaNextProcessor.from_pretrained(model_id)
model = LlavaNextForConditionalGeneration.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    low_cpu_mem_usage=True,
    device_map="auto" # GPU/CPU 자동 분산
)
model.eval()
```

```
# —— 프롬프트 구성 방법 (apply_chat_template 사용) ——

def build_llava_prompt(question: str) -> str:
    """
    LLaVA-1.6 Mistral 전용 채팅 템플릿 구성.
    이미지 위치는 <image> 토큰으로 지정됩니다.
    """
    conversation = [
        {
            "role": "user",
            "content": [
                {"type": "image"},          # 이미지 위치 지정
                {"type": "text", "text": question}, # 텍스트 질문
            ],
        }
    ]
    prompt = processor.apply_chat_template(
        conversation,
        add_generation_prompt=True
    )
    return prompt

# —— 다양한 프롬프트 예시 ——

prompt_examples = {
    "이미지 설명": "Please describe this image in detail.",
    "한국어 설명": "이 이미지에 대해 한국어로 자세히 설명해주세요.",
    "물체 나열": "List all the objects you can see in this image.",
    "장면 분석": "What is happening in this image? What is the mood?",
    "텍스트 추출": "Please transcribe all the text visible in this image.",
    "감성 분석": "What emotions or feelings does this image convey?",
    "비교 분석": "What are the main differences between the left and right parts of this image?",
    "스토리 생성": "Create a short story inspired by this image (3~4 sentences).",
    "제품 설명": "Describe this product as if you were writing an e-commerce listing.",
}

for task, question in prompt_examples.items():
    print(f"[{task}]: {build_llava_prompt(question)[:80]}...")
```

[코드 7-6] LLaVA-1.6 모델 로드 및 다양한 프롬프트 구성

```
# —— 이미지 + 텍스트 → 응답 생성 핵심 함수 ——
def generate_response(
    image: Image.Image,
    question: str,
    max_new_tokens: int = 512,
    temperature: float = 0.2,
    do_sample: bool = False
) -> str:
    """
    LLaVA 모델로 이미지에 대한 질문에 답변을 생성합니다.

    Args:
        image: PIL Image 객체
        question: 이미지에 대한 질문 (자연어)
        max_new_tokens: 생성할 최대 토큰 수
        temperature: 생성 다양성 (낮을수록 결정론적)
        do_sample: True이면 샘플링, False이면 greedy 디코딩

    Returns:
        모델의 응답 텍스트 문자열
    """
    prompt = build_llava_prompt(question)

    # 이미지 + 텍스트 프롬프트 → 모델 입력 형식으로 변환
    inputs = processor(images=image, text=prompt, return_tensors="pt")
    inputs = {k: v.to(model.device) for k, v in inputs.items()}

    with torch.inference_mode():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            temperature=temperature,
            do_sample=do_sample,
            pad_token_id=processor.tokenizer.eos_token_id,
        )

    # 입력 토큰 제외하고 생성된 부분만 디코딩
    input_len = inputs["input_ids"].shape[1]
```

```
generated_ids = output_ids[:, input_len:]
response = processor.decode(generated_ids[0], skip_special_tokens=True)
return response.strip()

# —— 실행 예시

image = Image.open("./sample_image.jpg").convert("RGB")

questions = [
    "What is shown in this image?",
    "이 이미지에서 보이는 것을 한국어로 설명해주세요.",
    "How many people are in this image?",
]

for q in questions:
    print(f"Q: {q}")
    answer = generate_response(image, q, max_new_tokens=200)
    print(f"A: {answer}")
```

[코드 7-7] 이미지+텍스트 → 응답 생성 핵심 함수 및 실행 예시

(3) Hugging Face Spaces에서 사용 가능한 경량화 모델 소개

LLaVA-7B 또는 MiniGPT-4-13B와 같은 대형 모델은 고사양 GPU(VRAM 14GB 이상)가 필요합니다. 제한된 환경에서도 멀티모달 챗봇을 구현하고 싶다면 경량화 모델이나 Hugging Face Spaces의 무료 GPU를 활용하는 방법이 있습니다.

모델명	VRAM 요구	파라미터	HF Hub ID	
moondream2	~ 3GB	1.8B	vikhyatk/moondream2	최경량. (가장 빠름)
LLaVA-1.5-7B (INT4)	~ 5GB	7B(양자화)	llava-hf/llava-1.5-7b-hf	양자화로 성능 향상
Idefics2-8B	~ 8GB	8B	HuggingFaceM4/idefics2-8b	다중 이미지 처리 가능
PaliGemma-3B	~ 6GB	3B	google/paligemma-3b-pt-224	Google의 최신 모델
Qwen2-VL-2B	~ 5GB	2B	Qwen/Qwen2-VL-2B-Instruct	알리바바의 멀티모달 모델

```
# — 경량 모델: moondream2 (최소 VRAM, CPU 가능) —
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
from PIL import Image
```

```
import torch
```

```
model_id = "vikhyatk/moondream2"
```

```
revision = "2024-08-26" # 버전 고정 (안정성)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id, revision=revision)
```

```
model = AutoModelForCausalLM.from_pretrained(
```

```
    model_id,
```

```
    trust_remote_code=True,
```

```
    revision=revision,
```

```
    torch_dtype=torch.float16,
```

```
    device_map={"": "cuda"} if torch.cuda.is_available() else None
```

```
)
```

```
model.eval()
```

```

def moondream_chat(image: Image.Image, question: str) -> str:
    """moondream2로 이미지에 대한 질문에 답변 생성"""
    enc_image = model.encode_image(image)
    answer = model.answer_question(
        enc_image,
        question,
        tokenizer,
        num_beams=4,
        no_repeat_ngram_size=3
    )
    return answer

image = Image.open("./sample.jpg").convert("RGB")
print(moondream_chat(image, "What do you see in this image?"))

# —— 양자화로 LLaVA-1.5-7B VRAM 절감 (bitsandbytes) ——
# pip install bitsandbytes

from transformers import LlavaForConditionalGeneration, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,          # 4bit 양자화 (INT4)
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)

model_4bit = LlavaForConditionalGeneration.from_pretrained(
    "llava-hf/llava-1.5-7b-hf",
    quantization_config=quantization_config,
    device_map="auto"
)

# INT4 양자화로 약 4~5GB VRAM에서 7B 모델 실행 가능
print("4bit 양자화 LLaVA 로드 완료")

```

[코드 7-8] 경량 moondream2 및 4bit 양자화 LLaVA 로드

(4) FastAPI로 간단한 챗봇 API 구성: 이미지 + 질문 → 설명 응답

멀티모달 챗봇을 FastAPI로 감싸서 이미지 파일과 텍스트 질문을 받아 자연어 응답을 반환하는 REST API를 구성합니다.

```
# multimodal_chat_api.py
from contextlib import asynccontextmanager
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from pydantic import BaseModel
from typing import Optional
import torch, io, time
from PIL import Image
from transformers import AutoModelForCausalLM, AutoTokenizer

# — 전역 모델 컨텍스트 —
mm_model = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # 앱 시작 시 moondream2 로드 (경량, CPU/GPU 모두 동작)
    print("멀티모달 모델 로딩 중...")
    model_id = "vikhyatk/moondream2"
    revision = "2024-08-26"
    device = "cuda" if torch.cuda.is_available() else "cpu"

    mm_model["tokenizer"] = AutoTokenizer.from_pretrained(
        model_id, revision=revision
    )
    mm_model["model"] = AutoModelForCausalLM.from_pretrained(
        model_id,
        trust_remote_code=True,
        revision=revision,
        torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    ).to(device)
    mm_model["model"].eval()
    mm_model["device"] = device
    print(f"모델 로드 완료 ({device})")
```



```

yield
mm_model.clear()

if torch.cuda.is_available():
    torch.cuda.empty_cache()

app = FastAPI(
    title="멀티모달 챗봇 API",
    description="이미지+텍스트를 함께 처리하는 멀티모달 AI API",
    version="1.0.0",
    lifespan=lifespan
)

class ChatResponse(BaseModel):
    answer: str
    question: str
    model: str
    processing_time_ms: float

@app.post("/api/v1/chat", response_model=ChatResponse)
async def multimodal_chat(
    image: UploadFile = File(..., description="이미지 파일 (JPG/PNG/WebP)",
    question: str = Form(..., description="이미지에 대한 질문 (자연어)",
    max_tokens: int = Form(default=300, ge=50, le=1000)
):
    """이미지 파일과 텍스트 질문을 받아 AI 응답을 반환합니다."""
    # 이미지 유효성 검사
    if not image.content_type.startswith("image/"):
        raise HTTPException(400, "이미지 파일만 업로드 가능합니다.")

    content = await image.read()
    if len(content) > 20 * 1024 * 1024: # 20MB 제한
        raise HTTPException(413, "파일 크기가 20MB를 초과합니다.")

    try:

```

```
    pil_image = Image.open(io.BytesIO(content)).convert("RGB")
except Exception:
    raise HTTPException(400, "이미지 파일을 파싱할 수 없습니다.")

start = time.time()
model = mm_model["model"]
tokenizer = mm_model["tokenizer"]

enc_image = model.encode_image(pil_image)
answer = model.answer_question(enc_image, question, tokenizer)

elapsed_ms = (time.time() - start) * 1000

return ChatResponse(
    answer=answer,
    question=question,
    model="moondream2",
    processing_time_ms=round(elapsed_ms, 2)
)
```

[코드 7-9] FastAPI 멀티모달 챗봇 API 전체 구현

7.3 FastAPI 기반 멀티모달 API 구성하기

(1) 이미지와 텍스트를 함께 POST로 보내는 방법

REST API에서 이미지 파일과 텍스트 데이터를 함께 전송하는 방법은 여러 가지가 있습니다. 멀티모달 API에서 가장 일반적으로 사용되는 방식은 multipart/form-data 방식이며, FastAPI는 이를 UploadFile과 Form의 조합으로 간편하게 처리합니다.

전송 방식	이미지 전달	텍스트 전달	특징	권장 상황
multipart/form-data	UploadFile (바이트리)	Form 필드	HTTP 표준, Swagger 지원	대부분의 멀티모달 API
JSON + base64	JSON 필드 (base64 인코딩)	JSON 필드	순수 JSON, 33% 크기 증가	모든 입출력을 JSON로 통일할 때
JSON + URL	JSON에 이미지 URL 포함	JSON 필드	서버가 URL에서 직접 다운로드	이미지가 이미 웹에 업로드된 경우
Streaming (WebSocket)	스트림 전송	스트림 전송	실시간, 저지연	실시간 카메라 피드

— 방법별 클라이언트 요청 예시 —

방법 1: multipart/form-data (권장)

```
import requests
```

```
with open("./image.jpg", "rb") as f:
```

```
    response = requests.post(
```

```
        "http://localhost:8000/api/v1/chat",
```

```
        files={"image": ("image.jpg", f, "image/jpeg")},
```

```
        data={"question": "이 이미지에 무엇이 보이나요?", "max_tokens": 300}
```

```
    )
```

```
print(response.json())
```

방법 2: base64 JSON

```
import base64, json
```

```

with open("./image.jpg", "rb") as f:
    b64_image = base64.b64encode(f.read()).decode("utf-8")

response = requests.post(
    "http://localhost:8000/api/v1/chat-json",
    json={
        "image_base64": b64_image,
        "image_type": "image/jpeg",
        "question": "이 이미지에 무엇이 보이나요?",
        "max_tokens": 300
    }
)

```

방법 3: 이미지 URL 전달

```

response = requests.post(
    "http://localhost:8000/api/v1/chat-url",
    json={
        "image_url": "https://example.com/photo.jpg",
        "question": "Describe this image.",
    }
)

```

```

# — curl 예시: multipart/form-data —————
# curl -X POST "http://localhost:8000/api/v1/chat" \
#   -F "image=@./photo.jpg;type=image/jpeg" \
#   -F "question=이 사진에서 무엇이 보이나요?" \
#   -F "max_tokens=300"

```

[코드 7-10] 이미지+텍스트 전송 방식별 클라이언트 코드 비교

(2) UploadFile + Form 조합 처리

FastAPI에서 파일과 폼 데이터를 함께 처리할 때는 UploadFile과 Form을 함수 파라미터로 동시에 선언합니다. 단, 이 경우 request body가 multipart/form-data가 되므로 JSON body 파라미터 (Pydantic 모델)와 혼용할 수 없다는 점에 주의해야 합니다.

```
# schemas.py - 멀티모달 API 요청/응답 스키마
from pydantic import BaseModel, Field
from typing import Optional, List

# JSON body 스키마 (base64 방식용)
class MultimodalJSONRequest(BaseModel):
    image_base64: str = Field(..., description="base64 인코딩된 이미지 데이터")
    image_type: str = Field(default="image/jpeg", description="이미지 MIME 타입")
    question: str = Field(..., min_length=1, max_length=1000)
    max_tokens: int = Field(default=300, ge=50, le=2000)
    temperature: float = Field(default=0.2, ge=0.0, le=2.0)
    stream: bool = Field(default=False, description="스트리밍 응답 여부")

class Config:
    json_schema_extra = {
        "example": {
            "image_base64": "base64_encoded_image_data_here...",
            "image_type": "image/jpeg",
            "question": "이 이미지에서 보이는 것을 설명해주세요.",
            "max_tokens": 300
        }
    }

# 응답 스키마
class MultimodalResponse(BaseModel):
    success: bool
    answer: str
    question: str
    model_id: str
    image_size: Optional[tuple] = None # (width, height)
    processing_time_ms: float
    tokens_generated: Optional[int] = None

class MultimodalBatchResponse(BaseModel):
    results: List[MultimodalResponse]
    total_time_ms: float
    success_count: int
```

```
fail_count: int
```

[코드 7-11] 멀티모달 API 요청/응답 Pydantic 스키마

```
# multimodal_router.py - 멀티모달 엔드포인트 모음
from fastapi import APIRouter, UploadFile, File, Form, HTTPException, Depends
from fastapi.responses import StreamingResponse
from typing import Optional, List
import io, base64, time, json
from PIL import Image

from schemas import MultimodalResponse, MultimodalJSONRequest

router = APIRouter(prefix="/api/v1", tags=["Multimodal"])

# ----- 엔드포인트 1: multipart/form-data (파일 업로드) -----
@router.post("/chat", response_model=MultimodalResponse,
            summary="멀티모달 채팅 (파일 업로드)",
            description="이미지 파일과 텍스트 질문을 함께 전송하여 AI 응답을 받습니다.")
)
async def chat_with_file(
    image: UploadFile = File(
        ...,
        description="JPG, PNG, WebP 형식의 이미지 파일"
    ),
    question: str = Form(
        ...,
        description="이미지에 대한 자연어 질문",
        example="이 이미지에서 무엇이 보이나요?"
    ),
    max_tokens: int = Form(default=300, ge=50, le=2000),
    temperature: float = Form(default=0.2, ge=0.0, le=2.0),
) -> MultimodalResponse:
    # 이미지 파일 검증
    allowed_types = {"image/jpeg", "image/png", "image/webp", "image/gif"}
    if image.content_type not in allowed_types:
        raise HTTPException(415, f"지원하지 않는 이미지 형식: {image.content_type}")
```

```

content = await image.read()
if len(content) > 20 * 1024 * 1024:
    raise HTTPException(413, "파일 크기가 20MB를 초과합니다.")

try:
    pil_image = Image.open(io.BytesIO(content)).convert("RGB")
except Exception:
    raise HTTPException(400, "이미지 파일 파싱에 실패했습니다.")

return await _run_inference(pil_image, question, max_tokens, temperature)

# ——— 엔드포인트 2: JSON + base64 ———
@router.post("/chat-json", response_model=MultimodalResponse,
    summary="멀티모달 채팅 (JSON/base64)"
)
async def chat_with_json(request: MultimodalJSONRequest) -> MultimodalResponse:
    try:
        image_bytes = base64.b64decode(request.image_base64)
        pil_image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    except Exception as e:
        raise HTTPException(400, f"base64 이미지 디코딩 실패: {str(e)}")

    return await _run_inference(
        pil_image, request.question, request.max_tokens, request.temperature
    )

# ——— 엔드포인트 3: URL에서 이미지 로드 ———
@router.post("/chat-url", response_model=MultimodalResponse,
    summary="멀티모달 채팅 (이미지 URL)"
)
async def chat_with_url(
    image_url: str = Form(..., description="공개 접근 가능한 이미지 URL"),
    question: str = Form(...)
) -> MultimodalResponse:
    import httpx

```

```
async with httpx.AsyncClient(timeout=10.0) as client:
    try:
        resp = await client.get(image_url)
        resp.raise_for_status()
    except Exception as e:
        raise HTTPException(400, f"이미지 URL 접근 실패: {str(e)}")

    pil_image = Image.open(io.BytesIO(resp.content)).convert("RGB")
    return await _run_inference(pil_image, question, 300, 0.2)
```

[코드 7-12] 3가지 입력 방식을 지원하는 멀티모달 라우터

(3) FormData → 모델 입력 전처리 파이프라인

업로드된 파일과 폼 데이터를 모델이 요구하는 입력 형식으로 변환하는 전처리 파이프라인을 체계적으로 구성합니다. 입력 검증, 이미지 전처리, 텍스트 정규화, 배치 조립의 4단계로 구성됩니다.

```
# preprocessing.py - 멀티모달 전처리 파이프라인
from PIL import Image
import io, re
from typing import Union
import torch

class MultimodalPreprocessor:
    """이미지+텍스트 입력의 전체 전처리 파이프라인을 담당합니다."""

    # 지원 이미지 형식
    ALLOWED_FORMATS = {"JPEG", "PNG", "WEBP", "GIF", "BMP"}
    MAX_FILE_SIZE = 20 * 1024 * 1024 # 20 MB
    TARGET_SIZE = (336, 336) # LLaVA 기본 입력 크기

    @staticmethod
    def validate_image_bytes(content: bytes) -> None:
        """이미지 바이트 데이터의 유효성을 검사합니다."""
        if len(content) == 0:
            raise ValueError("빈 파일이 업로드되었습니다.")
        if len(content) > MultimodalPreprocessor.MAX_FILE_SIZE:
            raise ValueError(f"파일 크기가 20MB를 초과합니다: {len(content)/1024/1024:.1f}MB")

    @staticmethod
    def load_image(content: bytes) -> Image.Image:
        """바이트 데이터를 PIL Image로 변환하고 기본 전처리를 수행합니다."""
        MultimodalPreprocessor.validate_image_bytes(content)

        try:
            image = Image.open(io.BytesIO(content))
        except Exception as e:
            raise ValueError(f"이미지 파싱 실패: {str(e)}")
```

```

# 형식 검증
if image.format not in MultimodalPreprocessor.ALLOWED_FORMATS:
    raise ValueError(f"지원하지 않는 이미지 형식: {image.format}")

# RGBA → RGB 변환 (투명 채널 제거)
if image.mode in ("RGBA", "P"):
    background = Image.new("RGB", image.size, (255, 255, 255))
    if image.mode == "P":
        image = image.convert("RGBA")
    background.paste(image, mask=image.split()[3])
    image = background
else:
    image = image.convert("RGB")

return image

@staticmethod
def resize_image(
    image: Image.Image,
    target_size: tuple = None,
    keep_aspect_ratio: bool = True,
    pad_to_square: bool = True
) -> Image.Image:
    """이미지를 모델 입력 크기로 리사이즈합니다."""
    if target_size is None:
        target_size = MultimodalPreprocessor.TARGET_SIZE

    if not keep_aspect_ratio:
        return image.resize(target_size, Image.LANCZOS)

    # 비율 유지 리사이즈
    image.thumbnail(target_size, Image.LANCZOS)

    if pad_to_square and image.size != target_size:
        # 흰색 배경으로 패딩하여 정사각형으로 만들기
        padded = Image.new("RGB", target_size, (255, 255, 255))

```

```

        offset = ((target_size[0] - image.width) // 2,
                  (target_size[1] - image.height) // 2)
        padded.paste(image, offset)
        return padded

    return image

    @staticmethod
    def normalize_question(question: str) -> str:
        """텍스트 질문을 정규화합니다."""
        # 앞뒤 공백 제거
        question = question.strip()
        # 연속 공백 단일화
        question = re.sub(r"Ws+", " ", question)
        # 최대 길이 초과 시 잘라내기
        if len(question) > 500:
            question = question[:500] + "..."
        return question

    def preprocess(
        self,
        image_bytes: bytes,
        question: str,
        resize: bool = True
    ) -> dict:
        """전체 전처리 파이프라인을 실행하고 정제된 입력을 반환합니다."""
        pil_image = self.load_image(image_bytes)
        original_size = pil_image.size

        if resize:
            pil_image = self.resize_image(pil_image)

        normalized_question = self.normalize_question(question)

        return {
            "image": pil_image,
            "question": normalized_question,

```

```
"original_size": original_size,  
"processed_size": pil_image.size,  
}
```

```
# 전처리기 싱글턴
```

```
preprocessor = MultimodalPreprocessor()
```

[코드 7-13] 멀티모달 전처리 파이프라인 클래스 (preprocessing.py)

(4) 이미지 추론 결과 + 텍스트 응답 결합 구조

추론 결과를 구조화된 응답으로 결합하고, 다양한 형식(JSON, 스트리밍 등)으로 반환하는 방법을 구현합니다. 특히 스트리밍(Server-Sent Events) 방식은 응답이 길 때 사용자 경험을 크게 향상시킵니다.

```
# inference_runner.py - 추론 실행 및 응답 결합
import asyncio, time
from concurrent.futures import ThreadPoolExecutor
from fastapi.responses import StreamingResponse
from PIL import Image
from schemas import MultimodalResponse

# 추론은 CPU/GPU 연산이므로 별도 스레드 풀에서 실행
_executor = ThreadPoolExecutor(max_workers=2)

def _sync_inference(pil_image: Image.Image, question: str,
                    max_tokens: int, temperature: float) -> dict:
    """동기식 모델 추론 (스레드에서 실행)"""
    from main import mm_model # 앱 전역 모델 참조

    start = time.time()
    enc_image = mm_model["model"].encode_image(pil_image)
    answer = mm_model["model"].answer_question(
        enc_image, question, mm_model["tokenizer"]
    )
    elapsed_ms = (time.time() - start) * 1000

    return {
        "answer": answer,
        "processing_time_ms": round(elapsed_ms, 2),
        "image_size": pil_image.size,
    }
```

```

async def _run_inference(
    pil_image: Image.Image,
    question: str,
    max_tokens: int = 300,
    temperature: float = 0.2
) -> MultimodalResponse:
    """비동기 래퍼: 스레드 풀에서 동기 추론을 실행합니다."""
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(
        _executor,
        _sync_inference,
        pil_image, question, max_tokens, temperature
    )
    return MultimodalResponse(
        success=True,
        answer=result["answer"],
        question=question,
        model_id="moondream2",
        image_size=result["image_size"],
        processing_time_ms=result["processing_time_ms"]
    )

```

——— 스트리밍 응답 엔드포인트 (Server-Sent Events) ———

```

from fastapi import APIRouter
router = APIRouter()

@router.post("/api/v1/chat/stream")
async def chat_stream(
    image: UploadFile = File(...),
    question: str = Form(...)
):
    content = await image.read()
    pil_image = Image.open(io.BytesIO(content)).convert("RGB")

    async def event_generator():
        # 시작 이벤트

```

```

yield f"data: {json.dumps({'status': 'processing', 'question': question})}\n\n"

# 추론 실행 (스트리밍은 별도 생성 구현 필요)
loop = asyncio.get_event_loop()
result = await loop.run_in_executor(
    _executor, _sync_inference, pil_image, question, 300, 0.2
)

# 응답 청크 전송 (단어 단위 시뮬레이션)
words = result["answer"].split()
for i, word in enumerate(words):
    chunk = word + (" " if i < len(words) - 1 else "")
    yield f"data: {json.dumps({'chunk': chunk, 'done': False})}\n\n"
    await asyncio.sleep(0.03) # 자연스러운 타이핑 효과

# 완료 이벤트
yield f"data: {json.dumps({'done': True, 'time_ms': result['processing_time_ms']})}\n\n"

return StreamingResponse(
    event_generator(),
    media_type="text/event-stream",
    headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"}
)

```

[코드 7-14] 비동기 추론 실행 및 SSE 스트리밍 응답 구현

(5) Swagger UI에서도 이미지 업로드 + 프롬프트 입력 지원 예시

FastAPI의 Swagger UI(/docs)는 UploadFile 파라미터를 자동으로 파일 업로드 버튼으로 렌더링합니다. 별도의 추가 설정 없이 이미지 업로드와 텍스트 입력을 Swagger UI에서 직접 테스트할 수 있습니다.

```

# — Swagger UI 최적화를 위한 FastAPI 앱 설정 —
from fastapi import FastAPI
from fastapi.openapi.docs import get_swagger_ui_html

```

```

app = FastAPI(
    title="멀티모달 AI API",
    description=""
)

## 멀티모달 AI API - 이미지 + 텍스트 통합 처리

### 지원 기능
- 🖼️ 이미지 파일 업로드 (JPG/PNG/WebP, 최대 20MB)
- 💬 자연어 질문으로 이미지 설명 요청
- 🔍 CLIP 기반 시맨틱 이미지 검색
- 🏠 Zero-shot 이미지 분류

### 사용법
1. `/api/v1/chat` 엔드포인트 선택
2. `Try it out` 버튼 클릭
3. 이미지 파일 선택 및 질문 입력
4. `Execute` 클릭하여 결과 확인

"""
,
version="1.0.0",
docs_url="/docs",
redoc_url="/redoc",
)

# — 엔드포인트에 상세 OpenAPI 문서 추가 —
@app.post(
    "/api/v1/chat",
    response_model=MultimodalResponse,
    summary="🗣️ 멀티모달 채팅",
    description=""
)
이미지와 텍스트 질문을 함께 받아 AI 응답을 반환합니다.

**요청 방식:** `multipart/form-data`

**이미지 형식:** JPG, PNG, WebP (최대 20MB)

**질문 예시:**
- "이 이미지에서 보이는 것을 설명해주세요."
- "이미지 속 사람이 몇 명인가요?"

```



```

- "이 제품의 특징을 설명해주세요."
"""
,
responses={
    200: {"description": "성공적으로 응답 생성"},
    400: {"description": "잘못된 이미지 파일"},
    413: {"description": "파일 크기 초과 (20MB 제한)"},
    415: {"description": "지원하지 않는 이미지 형식"},
    503: {"description": "모델 로딩 중 (잠시 후 재시도)"},
}
)
async def chat_endpoint(
    image: UploadFile = File(..., description="분석할 이미지 파일"),
    question: str = Form(..., example="이 이미지를 설명해주세요."),
    max_tokens: int = Form(default=300)
):
    ...

```

[코드 7-15] Swagger UI 최적화를 위한 OpenAPI 문서 설정

💡 Swagger UI에서 multipart/form-data 엔드포인트를 테스트할 때, Try it out → Execute를 클릭하면 자동으로 파일 업로드 인터페이스가 표시됩니다. 이미지를 선택하고 question 필드에 텍스트를 입력한 뒤 Execute를 누르면 즉시 테스트할 수 있습니다.

7.4 업로드 + 프롬프트 통합 처리

(1) 사용자 업로드 이미지 파일 → 임시 저장 or 메모리 처리

업로드된 이미지를 처리하는 방법은 크게 두 가지입니다. 파일로 저장하지 않고 메모리에서 직접 처리하는 In-Memory 방식과, 임시 파일로 저장한 후 처리하는 방식입니다. 각 방식은 용도와 환경에 따라 장단점이 뚜렷합니다.

처리 방식	구현 방법	장점	단점	
인메모리 처리	io.BytesIO → PIL Image 직접 변환	빠름, 디스크 I/O 없음, 보안(파일 없음)	메모리 사용량 증가, 대용량 불리	소~중규모
tempfile 임시 저장	tempfile.NamedTemporaryFile	라이브러리 호환성 우수, 경로 필요 시	디스크 I/O, 정리 필요	경로 필요
영구 저장 후 처리	os.makedirs + file.write	캐싱, 재사용, 로그 추적 가능	스토리지 관리 필요	감사 로그 시
스트리밍 처리	SpooledTemporaryFile	초대용량 이미지, 청크 처리	복잡한 구현	매우 큰 파일

```
# file_handler.py - 이미지 파일 처리 전략 구현
import io, os, uuid, tempfile, shutil
from contextlib import asynccontextmanager
from PIL import Image
from fastapi import UploadFile

#
# 방법 1: 인메모리 처리 (권장 - 대부분의 경우)
#
async def load_image_to_memory(upload: UploadFile) -> Image.Image:
    """업로드 파일을 메모리에서 직접 PIL Image로 변환합니다."""
    content = await upload.read()
    try:
        image = Image.open(io.BytesIO(content)).convert("RGB")
    except Exception as e:
```

```

        raise ValueError(f"이미지 파싱 실패: {e}")
    finally:
        await upload.seek(0) # 다시 읽을 수 있도록 포인터 리셋
    return image

#
# 방법 2: tempfile 임시 저장 (경로가 필요한 도구 사용 시)
#
@asynccontextmanager
async def save_as_tempfile(upload: UploadFile):
    """
    업로드 파일을 임시 파일로 저장하는 컨텍스트 매니저.
    with 블록 종료 시 임시 파일이 자동 삭제됩니다.

    Usage:
        async with save_as_tempfile(upload) as tmp_path:
            result = cv2.imread(tmp_path) # 경로가 필요한 경우
    """
    # 확장자 추출
    original_name = upload.filename or "upload"
    ext = os.path.splitext(original_name)[1] or ".tmp"

    # 임시 파일 생성
    tmp_file = tempfile.NamedTemporaryFile(
        delete=False,
        suffix=ext,
        prefix="upload_"
    )
    try:
        content = await upload.read()
        tmp_file.write(content)
        tmp_file.flush()
        tmp_file.close()
        yield tmp_file.name # 임시 파일 경로 반환
    finally:
        # 컨텍스트 종료 시 임시 파일 삭제
        if os.path.exists(tmp_file.name):

```

```

os.unlink(tmp_file.name)

#
# 방법 3: 영구 저장 (캐싱/감사 로그 필요 시)
#
UPLOAD_DIR = "./uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

async def save_upload_permanently(
    upload: UploadFile,
    subdirectory: str = ""
) -> str:
    """
    업로드 파일을 영구적으로 저장하고 저장 경로를 반환합니다.
    고유 UUID 파일명으로 충돌을 방지합니다.
    """
    ext = os.path.splitext(upload.filename or "img")[1] or ".bin"
    unique_name = f"{uuid.uuid4().hex}{ext}"

    save_dir = os.path.join(UPLOAD_DIR, subdirectory)
    os.makedirs(save_dir, exist_ok=True)
    save_path = os.path.join(save_dir, unique_name)

    content = await upload.read()
    with open(save_path, "wb") as f:
        f.write(content)

    return save_path

# ——— 사용 예시 비교 ———
# 방법 1: 인메모리 (권장)
# image = await load_image_to_memory(upload)
# result = model_inference(image)

```

```
# 방법 2: 임시 파일 (경로 필요 시)
# async with save_as_tempfile(upload) as path:
#     img_array = cv2.imread(path)
#     result = cv2_based_inference(img_array)

# 방법 3: 영구 저장
# path = await save_upload_permanently(upload, subdirectory="2025/01")
# log_upload(user_id, path)
```

[코드 7-16] 이미지 업로드 처리 3가지 방법 구현 (file_handler.py)

(2) 프롬프트는 자연어 그대로 처리 (예: "이 그림은 무슨 내용이야?")

사용자가 입력하는 자연어 프롬프트는 한국어든 영어든 그대로 처리할 수 있도록 설계해야 합니다. 단, 언어 모델마다 최적의 언어가 다를 수 있으므로 필요에 따라 언어 감지 및 번역 옵션을 제공하면 사용자 경험을 높일 수 있습니다.

```
# prompt_handler.py - 자연어 프롬프트 처리
from typing import Optional
import re

# ——— 프롬프트 전처리 ———
def clean_prompt(text: str) -> str:
    """사용자 입력 프롬프트를 정제합니다."""
    # 앞뒤 공백 제거
    text = text.strip()
    # 연속 공백 정리
    text = re.sub(r"\\s+", " ", text)
    # 최대 길이 제한 (CLIP 등 max 77 tokens, LLaVA max ~500 chars)
    if len(text) > 800:
        text = text[:800].rsplit(" ", 1)[0] # 단어 중간에서 자르지 않도록
    return text

# ——— 간단한 언어 감지 (langdetect 패키지 불필요) ———
def detect_language_simple(text: str) -> str:
    """간단한 규칙 기반 언어 감지 (한국어 vs 영어)"""
    # 한글 유니코드 범위: AC00-D7A3 (가-힣), 1100-11FF, 3130-318F
    korean_chars = sum(1 for c in text if "\\uAC00" <= c <= "\\uD7A3"
                        or "\\u1100" <= c <= "\\u11FF")
    korean_ratio = korean_chars / max(len(text), 1)
    return "ko" if korean_ratio > 0.15 else "en"

# ——— 프롬프트 의도 분류 ———
INTENT_PATTERNS = {
```

```

"describe": ["설명", "describe", "tell me about", "what is", "무엇", "어떤", "what do you see"],
"count": ["몇", "how many", "count", "number of"],
"identify": ["누구", "who", "identify", "which"],
"ocr": ["텍스트", "글자", "읽어", "text", "read", "transcribe", "written"],
"compare": ["비교", "compare", "difference", "차이", "같은", "다른"],
"sentiment": ["감정", "느낌", "mood", "emotion", "feel"],
}

def classify_intent(question: str) -> str:
    """질문의 의도를 분류합니다."""
    question_lower = question.lower()
    for intent, keywords in INTENT_PATTERNS.items():
        if any(kw in question_lower for kw in keywords):
            return intent
    return "general"

# — 언어별 시스템 프롬프트 선택 —
SYSTEM_PROMPTS = {
    "en": "You are a helpful visual AI assistant. Analyze the image carefully and provide accurate, detailed answers.",
    "ko": "당신은 도움이 되는 시각적 AI 어시스턴트입니다. 이미지를 주의 깊게 분석하고 정확하고 상세한 답변을 제공하세요.",
}

def build_full_prompt(question: str, include_system: bool = True) -> dict:
    """전처리된 프롬프트 정보와 감지된 언어를 반환합니다."""
    cleaned = clean_prompt(question)
    lang = detect_language_simple(cleaned)
    intent = classify_intent(cleaned)
    system = SYSTEM_PROMPTS.get(lang, SYSTEM_PROMPTS["en"])

    return {
        "original": question,
        "cleaned": cleaned,
        "language": lang,
        "intent": intent,
        "system_prompt": system if include_system else None,
    }

```

```

    }

# —— 테스트

test_questions = [
    "이 그림은 무슨 내용이야?",
    "How many people are in this photo?",
    "사진 속 텍스트를 읽어줘",
    "Describe the mood of this image",
]

for q in test_questions:
    info = build_full_prompt(q)
    print(f"[{info['language']:2s}][{info['intent']:10s}] {q}")

```

[코드 7-17] 자연어 프롬프트 전처리 및 의도 분류 (prompt_handler.py)

(3) 통합 처리 흐름: 업로드 → 전처리 → 모델 추론 → 응답

지금까지 다룬 모든 구성 요소를 통합하여 완전한 멀티모달 API 서버를 구성합니다. 이미지 업로드 처리, 텍스트 전처리, 모델 추론, 응답 구조화까지의 전체 흐름을 하나의 완성된 코드로 구현합니다.

🔗 완전한 멀티모달 API 처리 흐름 (5단계)

① HTTP 요청 수신: POST /api/v1/chat

multipart/form-data: image 파일 + question 텍스트

② 입력 검증 (Validation):

- 파일 크기 검사 ($\leq 20\text{MB}$)
- MIME 타입 검증 (image/jpeg, image/png, image/webp)
- 텍스트 길이 검사 (1 ~ 800자)

③ 전처리 (Preprocessing):

- 이미지: BytesIO → PIL Image → RGB 변환 → 336×336 리사이즈
- 텍스트: 공백 정리 → 언어 감지 → 의도 분류 → 시스템 프롬프트 결합

④ 모델 추론 (Inference):

- 비동기 스레드 풀에서 동기 추론 실행 (이벤트 루프 블로킹 방지)
- 이미지 인코딩 → 텍스트 임베딩 결합 → LLM 생성

⑤ 응답 구조화 및 반환:

- MultimodalResponse Pydantic 모델로 직렬화
- 처리 시간, 이미지 메타데이터, 언어 정보 포함

```
# complete_api.py - 모든 구성 요소 통합 완전한 멀티모달 API 서버
```

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional
import torch, io, time, asyncio
from concurrent.futures import ThreadPoolExecutor
from PIL import Image
```

```
# 자체 모듈 임포트
```

```
from file_handler import load_image_to_memory
from prompt_handler import build_full_prompt, clean_prompt
from preprocessing import MultimodalPreprocessor
```

```
# — 전역 상태
```

```
app_state = {}
_executor = ThreadPoolExecutor(max_workers=2)
preprocessor = MultimodalPreprocessor()
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
```

```
"""앱 시작/종료 시 모델 초기화 및 정리"""
print("=" * 50)
print("멀티모달 API 서버 시작 중...")

# 장치 선택
device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32
print(f"실행 장치: {device}")

from transformers import AutoModelForCausalLM, AutoTokenizer
model_id = "vikhyatk/monodream2"
revision = "2024-08-26"

app_state["tokenizer"] = AutoTokenizer.from_pretrained(
    model_id, revision=revision)
app_state["model"] = AutoModelForCausalLM.from_pretrained(
    model_id, trust_remote_code=True, revision=revision,
    torch_dtype=dtype).to(device)
app_state["model"].eval()
app_state["device"] = device
print("모델 로드 완료")
print("=" * 50)

yield

app_state.clear()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
print("서버 종료 완료")

app = FastAPI(
    title="멀티모달 AI API v1.0",
    description="이미지+텍스트 통합 멀티모달 AI 서비스",
    version="1.0.0",
    lifespan=lifespan
)
```

```
# CORS 설정 (프론트엔드 연동 시 필요)
```

```
app.add_middleware(
```

```
    CORSMiddleware,
```

```
    allow_origins=["*"],
```

```
    allow_methods=["*"],
```

```
    allow_headers=["*"],
```

```
)
```

```
# —— 응답 스키마
```

```
class FullMultimodalResponse(BaseModel):
```

```
    success: bool
```

```
    answer: str
```

```
    question_original: str
```

```
    question_cleaned: str
```

```
    detected_language: str
```

```
    detected_intent: str
```

```
    image_original_size: list
```

```
    image_processed_size: list
```

```
    model_id: str
```

```
    processing_time_ms: float
```

```
# —— 핵심 추론 함수 (동기, 스레드에서 실행) ——
```

```
def _run_model(image: Image.Image, question: str) -> str:
```

```
    """모델 추론 실행 (동기 함수 - 스레드에서 호출됨)"""
```

```
    model = app_state["model"]
```

```
    tokenizer = app_state["tokenizer"]
```

```
    enc = model.encode_image(image)
```

```
    return model.answer_question(enc, question, tokenizer)
```

```
# —— 메인 멀티모달 엔드포인트 ——
```

```
@app.post("/api/v1/chat",
```

```
    response_model=FullMultimodalResponse,
```

```

summary="멀티모달 채팅 API (통합)",
)
async def full_multimodal_chat(
    image: UploadFile = File(..., description="분석할 이미지 (JPG/PNG/WebP)"),
    question: str = Form(..., description="이미지에 대한 자연어 질문",
                        example="이 이미지에서 무엇이 보이나요?"),
    max_tokens: int = Form(default=300, ge=50, le=2000)
):
    # ① 모델 준비 확인
    if "model" not in app_state:
        raise HTTPException(503, "모델이 아직 로드되지 않았습니다. 잠시 후 다시 시도해주세요.")

    # ② 이미지 로드 (인메모리)
    content = await image.read()
    if len(content) > 20 * 1024 * 1024:
        raise HTTPException(413, "파일 크기가 20MB를 초과합니다.")

    try:
        pil_image = Image.open(io.BytesIO(content)).convert("RGB")
        original_size = list(pil_image.size)
    except Exception:
        raise HTTPException(400, "이미지 파일을 열 수 없습니다.")

    # ③ 이미지 전처리
    processed = preprocessor.resize_image(pil_image)
    processed_size = list(processed.size)

    # ④ 텍스트 전처리
    prompt_info = build_full_prompt(question)

    # ⑤ 비동기 추론 (스레드 풀)
    start = time.time()
    loop = asyncio.get_event_loop()
    try:
        answer = await loop.run_in_executor(
            _executor,
            _run_model,
            processed,

```

```

        prompt_info["cleaned"]
    )
except Exception as e:
    raise HTTPException(500, f"모델 추론 오류: {str(e)}")

elapsed_ms = (time.time() - start) * 1000

# ⑥ 응답 구조화
return FullMultimodalResponse(
    success=True,
    answer=answer,
    question_original=question,
    question_cleaned=prompt_info["cleaned"],
    detected_language=prompt_info["language"],
    detected_intent=prompt_info["intent"],
    image_original_size=original_size,
    image_processed_size=processed_size,
    model_id="moondream2",
    processing_time_ms=round(elapsed_ms, 2)
)

# — 헬스체크

@app.get("/health")
async def health_check():
    return {
        "status": "healthy",
        "model_loaded": "model" in app_state,
        "device": app_state.get("device", "unknown")
    }

```

[코드 7-18] 통합 완전한 멀티모달 API 서버 (complete_api.py)

(4) Swagger UI, Postman, Streamlit 등에서 테스트 방법

구현된 멀티모달 API를 다양한 도구로 테스트하는 방법을 안내합니다. 각 도구는 목적과 사용 편의성이 달라 개발 단계에 따라 적절히 선택하여 사용합니다.

```
# —— 1. 서버 실행 ——  
uvicorn complete_api:app --host 0.0.0.0 --port 8000 --reload  
  
# 접속 주소:  
# Swagger UI: http://localhost:8000/docs  
# ReDoc:      http://localhost:8000/redoc  
# Health:     http://localhost:8000/health  
  
# —— 2. curl 테스트 ——  
# multipart/form-data 방식 (파일 업로드)  
curl -X POST "http://localhost:8000/api/v1/chat" \\  
-F "image=@./sample_photo.jpg;type=image/jpeg" \\  
-F "question=이 이미지에서 보이는 것을 한국어로 설명해주세요." \\  
-F "max_tokens=300" | python3 -m json.tool  
  
# —— 3. Python requests 클라이언트 ——  
import requests, json  
  
API_URL = "http://localhost:8000/api/v1/chat"  
  
def test_multimodal_api(image_path: str, question: str) -> dict:  
    with open(image_path, "rb") as f:  
        files = {"image": (image_path, f, "image/jpeg")}  
        data = {"question": question, "max_tokens": 300}  
        response = requests.post(API_URL, files=files, data=data, timeout=60)  
        response.raise_for_status()  
        return response.json()
```

```
# 테스트 실행
result = test_multimodal_api("./photo.jpg", "이 이미지를 설명해주세요.")
print(json.dumps(result, ensure_ascii=False, indent=2))
```

[코드 7-19] 서버 실행 및 curl/Python 클라이언트 테스트

```
# — 4. Streamlit 멀티모달 테스트 앱 —————
# streamlit_test.py
# 실행: streamlit run streamlit_test.py

import streamlit as st
import requests, json
from PIL import Image
import io

st.set_page_config(page_title="멀티모달 AI 테스트", page_icon="🤖", layout="wide")
st.title("🤖 멀티모달 AI API 테스트")

API_BASE = st.sidebar.text_input("API 서버 주소", value="http://localhost:8000")

col_left, col_right = st.columns([1, 1])

with col_left:
    st.subheader("📁 입력")
    uploaded_file = st.file_uploader(
        "이미지 파일 선택", type=["jpg", "jpeg", "png", "webp"]
    )

    if uploaded_file:
        image = Image.open(uploaded_file)
        st.image(image, caption=f"업로드: {uploaded_file.name}", use_column_width=True)

    question = st.text_area(
        "질문 입력",
        value="이 이미지에서 보이는 것을 자세히 설명해주세요.",
        height=100
    )
```

```

max_tokens = st.slider("최대 토큰 수", 50, 1000, 300)

if st.button("🔍 분석 요청", type="primary", disabled=uploaded_file is None):
    with st.spinner("AI 분석 중..."):
        uploaded_file.seek(0)
        files = {"image": (uploaded_file.name, uploaded_file.getvalue(), "image/jpeg")}
        data = {"question": question, "max_tokens": max_tokens}
        try:
            resp = requests.post(f"{API_BASE}/api/v1/chat",
                                files=files, data=data, timeout=120)
            if resp.status_code == 200:
                st.session_state["result"] = resp.json()
            else:
                st.error(f"오류 {resp.status_code}: {resp.text}")
        except requests.exceptions.ConnectionError:
            st.error("API 서버에 연결할 수 없습니다. 서버가 실행 중인지 확인하세요.")

with col_right:
    st.subheader("📄 응답")
    if "result" in st.session_state:
        r = st.session_state["result"]
        st.success("분석 완료!")

    st.markdown("### 🤖 AI 답변")
    st.write(r["answer"])

    with st.expander("📊 상세 정보"):
        col_a, col_b = st.columns(2)
        col_a.metric("처리 시간", f"{r['processing_time_ms']:.1f} ms")
        col_b.metric("감지 언어", r.get("detected_language", "N/A"))
        st.json({k: v for k, v in r.items() if k != "answer"})

    if st.button("📋 결과 복사"):
        st.code(r["answer"])

```

[코드 7-20] Streamlit 멀티모달 API 테스트 앱

— 5. Postman 설정 가이드 (텍스트 형태로 설명) —


```

#
# [Postman에서 멀티모달 API 테스트 방법]
#
# 1. New Request 생성
#   - Method: POST
#   - URL: http://localhost:8000/api/v1/chat
#
# 2. Body 탭 선택 → form-data 선택
#
# 3. 필드 추가:
#   KEY      TYPE   VALUE
#   -----
#   image     File   [파일 선택] → 이미지 파일 업로드
#   question  Text    이 이미지를 설명해주세요.
#   max_tokens Text    300
#
# 4. Send 클릭 → Response Body에서 JSON 결과 확인
#
# 5. Tests 탭에서 자동 검증 스크립트 추가 (선택):
pm.test("Status is 200", () => pm.response.to.have.status(200));
pm.test("Has answer field", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("answer");
  pm.expect(json.answer).to.be.a("string");
  pm.expect(json.answer.length).to.be.above(0);
});
pm.test("Processing time reasonable", () => {
  const json = pm.response.json();
  pm.expect(json.processing_time_ms).to.be.below(30000);
});

# — 6. pytest 기반 자동화 테스트 —
# test_api.py
import pytest
from fastapi.testclient import TestClient
from complete_api import app
import io
from PIL import Image

```

```
client = TestClient(app)

def make_test_image_bytes(color=(100, 150, 200)) -> bytes:
    """테스트용 임시 이미지 바이트 생성"""
    img = Image.new("RGB", (224, 224), color=color)
    buf = io.BytesIO()
    img.save(buf, format="JPEG")
    return buf.getvalue()

def test_health_check():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "healthy"

def test_multimodal_chat_basic():
    image_bytes = make_test_image_bytes()
    response = client.post(
        "/api/v1/chat",
        files={"image": ("test.jpg", image_bytes, "image/jpeg")},
        data={"question": "What color is this image?", "max_tokens": 100}
    )
    assert response.status_code == 200
    data = response.json()
    assert data["success"] is True
    assert len(data["answer"]) > 0
    assert data["processing_time_ms"] > 0

def test_invalid_file_type():
    response = client.post(
        "/api/v1/chat",
        files={"image": ("test.txt", b"hello world", "text/plain")},
        data={"question": "test"}
    )
    assert response.status_code in (400, 415)

# 실행: pytest test_api.py -v
```

[코드 7-21] Postman 설정 가이드 및 pytest 자동화 테스트

7장 마무리

이 장에서는 멀티모달 AI의 핵심 기술인 CLIP, LLaVA, MiniGPT-4를 실습하고, 이를 FastAPI 기반의 완전한 웹 API로 구현하는 전 과정을 다루었습니다. 텍스트와 이미지를 함께 처리하는 멀티모달 AI 서비스의 기초를 완성했습니다.

각 절의 핵심 학습 내용을 정리합니다.

섹션	핵심 기술	실무 적용 포인트
7.1 CLIP	공동 임베딩 공간, 코사인 유사도, Zero-shot 분류	시맨틱 이미지 검색 엔진, 이미지 태깅 자동화
7.2 LLaVA/챗봇	멀티모달 LLM, 비전 인코더 + LLM 연결	이미지 설명 자동화, 시각적 QA 시스템
7.3 멀티모달 API	UploadFile + Form, 3가지 입력 방식, SSE 스트리밍	프로덕션 수준 멀티모달 REST API 설계
7.4 통합 처리	인메모리/임시파일/영구 저장, 프롬프트 처리	엔드-투-엔드 멀티모달 서비스 파이프라인

🔗 다음 장 미리보기 - 8장: AI 서비스 배포 및 운영

8장에서는 개발한 AI 서비스를 실제 프로덕션 환경에 배포하고 운영하는 방법을 다룹니다.

- Docker + Docker Compose로 AI 서비스 컨테이너화
- AWS/GCP/Azure GPU 인스턴스에 배포하기
- Nginx + HTTPS 설정으로 프로덕션 서버 구성
- 모니터링: Prometheus + Grafana로 AI API 성능 추적
- 스케일링: 여러 모델 인스턴스를 로드 밸런서로 분산

연습 문제

1. CLIP openai/clip-vit-base-patch32 모델을 로드하고, 10장 이상의 이미지를 CLIPImageSearcher 클래스로 인덱싱한 뒤, 다음 텍스트 쿼리로 각각 검색하여 결과를 비교하세요: (A) "a cat sitting on a sofa" (B) "outdoor scenery with trees". 가장 높은 유사도 점수와 가장 낮은 점수의 차이를 분석하세요.
2. 7.1절의 zero_shot_classify 함수를 사용하여 임의의 이미지 5장을 ["indoor scene", "outdoor scene", "animal", "person", "food", "vehicle"] 중 하나로 분류하는 코드를 작성하고, 각 분류의 확신도 점수와 실제 이미지 내용을 비교하세요.
3. 7.2절의 FastAPI 멀티모달 챗봇 API(multimodal_chat_api.py)를 실행하고, Swagger UI(/docs)에서 직접 이미지를 업로드하여 다음 질문들로 테스트하세요: (A) "이 이미지를 한국어로 설명해주세요." (B) "What objects do you see?" (C) "How many distinct colors are in this image?"
4. 7.3절의 MultimodalPreprocessor 클래스를 확장하여, 이미지의 너비 또는 높이가 4096픽셀을 초과하는 경우 자동으로 비율을 유지하면서 1024px 이하로 리사이즈하는 기능을 추가하고, EXIF 데이터에서 회전 정보를 추출하여 자동 회전 보정하는 기능도 구현하세요.
5. (심화) CLIP 이미지 검색 기능과 LLaVA 챗봇 기능을 통합하는 API를 설계하세요. 사용자가 텍스트 쿼리를 입력하면 CLIP으로 가장 유사한 이미지를 찾고, 찾아진 이미지를 LLaVA에 전달하여 상세한 설명을 생성하는 "검색 + 설명" 파이프라인을 구현하세요.

— 7장 끝 —